

Arduino 1 — Install the Arduino IDE and Run the Blink Test

Tutorial Group: Arduino

Goal

Install and configure the Arduino IDE for the Arduino Mega 2560, connect the Arduino to your computer, and successfully run the built-in **Blink** tutorial.

This tutorial also introduces the basic structure of an Arduino program:

- `setup()`
- `loop()`
- `delay()`
- Digital outputs

By the end of this tutorial, you should be able to install the Arduino IDE, upload a program to the Mega 2560, understand the basic structure of an Arduino program, and make a change to a program and observe the result.

What You Need

- Arduino Mega 2560
- USB cable suitable for your Mega 2560
- Computer running Windows, macOS, or Linux
- Internet connection
- **No external components are required for this tutorial.**
The Mega 2560 has a built-in LED that will be used for this tutorial.

Part 1 — Install the Arduino IDE

Download and install the current Arduino IDE from the official Arduino website. The instructions on how to install the IDE and the latest version for each development system (Windows, macOS, or Linux) can be found here:

<https://support.arduino.cc/hc/en-us/articles/360019833020-Download-and-install-Arduino-IDE>

During installation, allow the installer to install any required USB drivers if prompted. After installation, start the **Arduino IDE**. You should see a window containing an Arduino program.

Arduino programs are commonly called **sketches**. A basic Arduino sketch has two special functions:

```
void setup() {  
}  
  
void loop() {  
}
```

We will explain these functions shortly.

Part 2 — Connect the Arduino Mega 2560

1. Connect the USB cable between the Arduino Mega 2560 and your computer.
2. Plug the USB cable directly into the computer.
3. The Arduino should power up.
4. Look for the power LED on the Arduino board.

The USB connection provides both:

- Power to the Arduino
- A communications connection between the Arduino and your computer

Part 3 — Select the Arduino Mega 2560

In the Arduino IDE, select the Arduino Mega 2560 as the target board.

Depending on the version of the Arduino IDE, this may be found under:

Tools → Board → Arduino AVR Boards → Arduino Mega 2560 or Mega 2560

If the IDE asks for a processor, select:

ATmega2560

Part 4 — Select the USB/Serial Port

The Arduino IDE needs to know which computer port is connected to your Arduino.

Look under:

Tools → Port

You should see one or more available ports.

A useful way to identify the correct port is:

1. Disconnect the Arduino USB cable.
2. Look at the available ports.
3. Reconnect the Arduino.
4. Look at the available ports again.
5. The newly appearing port should be the Arduino.

Select that port.

If no port appears, don't immediately assume the Arduino is defective.

Check:

- Is the USB cable connected?
- Is the cable capable of carrying data? Some USB cables are power-only.
- Is the Arduino powered?
- Does another USB port work?
- Does the computer recognize the device?
- Did the Arduino IDE install the required driver?

If you are using an inexpensive Mega 2560-compatible board rather than an official Arduino Mega 2560, the USB interface may use a different USB-to-serial chip and may require a different driver.

Part 5 — Open the Blink Tutorial

The Arduino IDE includes many example sketches of the Arduino and various sensors.

Open the built-in Blink tutorial:

File → Examples → 01.Basics → Blink

A new editor window or tab should open containing the Blink program.

You do **not** need to type the program yourself.

The Blink tutorial uses the Arduino's built-in LED.

Part 6 — Understanding setup() and loop()

Before uploading the program, it is important to understand the basic structure of an Arduino program.

Most Arduino programs contain two special functions:

```
void setup() {  
    // setup code goes here  
}  
  
void loop() {  
    // repeating code goes here  
}
```

The code inside setup() is executed **once** when the Arduino starts running the program.

It is also executed again when the Arduino is reset.

Things that only need to happen once generally belong in setup().

Examples of code in setup() include:

- Configuring input and output pins
- Starting Serial communication
- Initializing a display
- Setting initial values

Think of setup() as getting everything ready to run.

After setup() has finished, the Arduino begins executing loop().

When it reaches the end of loop(), it immediately starts again at the beginning of loop().

It continues doing this **forever**, as long as the Arduino is powered or until it is reset.

For example:

```
void loop() {  
    // Step 1  
    // Step 2  
    // Step 3  
}
```

The Arduino executes:

```
Step 1  
Step 2  
Step 3  
Step 1  
Step 2  
Step 3  
...
```

Without anything slowing it down, the loop executes as fast as the processor can execute the instructions.

Part 7 — Slowing Down the Loop with delay()

One simple way to control how quickly the program repeats is the Arduino `delay()` function.

For example:

```
delay(1000);
```

tells the Arduino to wait approximately **1000 milliseconds**, or **1 second**, before continuing.

For example:

```
void loop() {  
    // Do something  
    delay(1000);  
  
    // Do something else  
    delay(1000);  
}
```

The Arduino performs the first action, waits one second, performs the second action, waits one second, and then starts `loop()` again.

An important note about delay()

`delay()` is a simple way to control timing, and it is useful while learning Arduino programming.

However, `delay()` also means that the Arduino is essentially waiting and cannot easily perform other tasks during that time.

We will use `delay()` in these first tutorials because it makes the operation of the program easy to understand. Later tutorials will introduce better techniques for controlling timing when the Arduino needs to perform other tasks at the same time.

Part 8 — Look at the Blink Program

The Blink tutorial should contain code similar to:

```
void setup() {  
    pinMode(LED_BUILTIN, OUTPUT);  
}  
  
void loop() {  
    digitalWrite(LED_BUILTIN, HIGH);  
    delay(1000);  
    digitalWrite(LED_BUILTIN, LOW);  
    delay(1000);  
}
```

Don't worry about understanding every part of the program yet.

There are two important things to notice.

The setup() function

```
pinMode(LED_BUILTIN, OUTPUT);
```

This tells the Arduino that the built-in LED is going to be used as an **output**.

This only needs to be done once, so it belongs in `setup()`.

The `loop()` function

```
digitalWrite(LED_BUILTIN, HIGH);  
delay(1000);  
  
digitalWrite(LED_BUILTIN, LOW);  
delay(1000);
```

The Arduino:

1. Turns the LED on.
2. Waits one second.
3. Turns the LED off.
4. Waits one second.
5. Starts `loop()` again.

The result is a continuously blinking LED.

Part 9 — Compile the Program

Before sending the program to the Arduino, the IDE must compile it.

Click the **Verify** button (the checkmark).

The IDE should compile the program without errors.

You should see a message indicating that compilation was successful.

What does “compile” mean?

The program you see is written in a programming language that is understandable to humans.

The Arduino itself does not directly execute that source code.

The Arduino IDE converts the program into instructions that the Mega 2560’s processor can execute.

This conversion process is called **compiling**.

Part 10 — Upload the Program

Once the program compiles successfully:

1. Make sure the Mega 2560 is still connected.
2. Make sure the correct board is selected.
3. Make sure the correct port is selected.
4. Click the **Upload** button (the right-arrow).

The IDE will compile the program and then transfer it to the Arduino.

During the upload, you may see the LEDs on the Arduino flash or change state.

When the upload is complete, the Arduino should automatically begin running the program.

Part 11 — Verify the Blink Test

Look at the built-in LED on the Arduino Mega 2560.

It should repeatedly:

Turn ON → wait → turn OFF → wait → repeat

with approximately a one-second interval between changes.

If the LED is blinking, your computer has successfully compiled and transferred a program to the Arduino.

Part 12 — Experiment with the Program

Now that the original program works, make a copy of the sketch and experiment with it.

Find these lines:

```
delay(1000);
```

There are two of them.

Change both values to:

```
delay(500);
```

Compile and upload the program again.

The LED should now blink approximately twice as fast.

Try other values.

For example:

```
100 ms  
250 ms  
500 ms  
1000 ms  
2000 ms
```

Observe how the value of `delay()` affects the blink rate.

Part 13 — Make the Blink Asymmetrical

Now try something slightly different.

Change the two delays to different values.

For example:

```
digitalWrite(LED_BUILTIN, HIGH);  
delay(250);  
  
digitalWrite(LED_BUILTIN, LOW);  
delay(1000);
```

The LED should now:

- Stay on for approximately ¼ second.
- Stay off for approximately 1 second.

This demonstrates that the Arduino isn't simply "blinking."

It is executing a sequence of instructions:

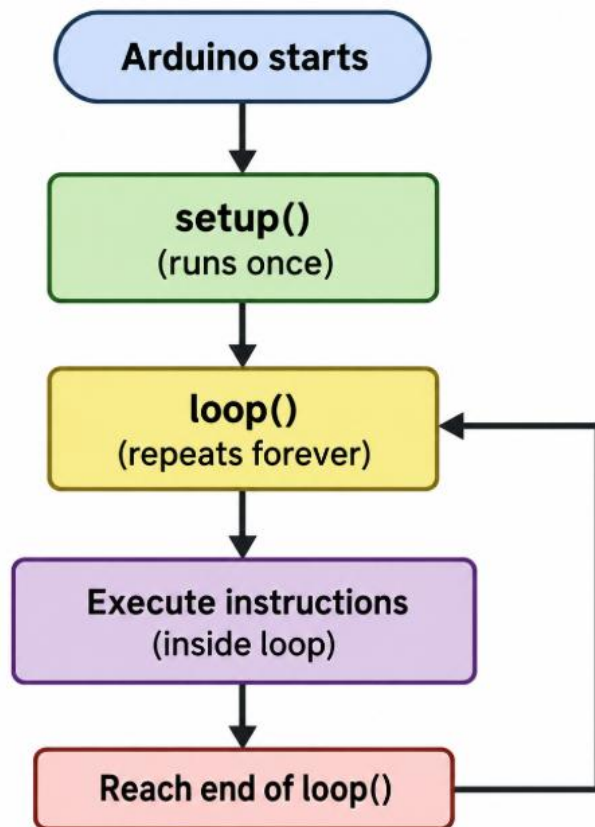
```
Turn ON
```

Wait
Turn OFF
Wait
Repeat

The program determines exactly what happens and when it happens.

What You Should Understand

You have now learned the basic execution model used by Arduino programs:



You also learned that:

- setup() runs once when the Arduino starts or resets.
- loop() repeats continuously.
- delay() pauses the program for a specified amount of time.
- pinMode() configures a pin
- digitalWrite() changes a digital output.
- The Arduino can control a physical device based on the instructions in your program.

These concepts will be used throughout the remaining tutorials.

Before Moving On

You should be able to answer **yes** to all of the following:

- ☐ I installed the Arduino IDE.

- ☐ I connected an Arduino Mega 2560 to my computer.
- ☐ I selected the Arduino Mega 2560 in the IDE.
- ☐ I selected the correct USB/serial port.
- ☐ I opened the built-in Blink tutorial.
- ☐ I understand that `setup()` runs once when the Arduino starts or resets.
- ☐ I understand that `loop()` repeats continuously.
- ☐ I understand that `delay()` pauses program execution.
- ☐ I successfully compiled the Blink program.
- ☐ I successfully uploaded the program.
- ☐ I verified that the built-in LED blinks.
- ☐ I changed the delay values and observed the blink rate change.
- ☐ I changed the two delay values independently and observed the effect.

If you can complete all of these tasks, you are ready for the next tutorial.